Trie, Some Graph Algorithms, and Dynamic Programming

Saptarshi Pyne, Assistant Professor
Department of Computational and Data Sciences (CDS)
Indian Institute of Science Education and Research (IISER) Kolkata
Mohanpur, West Bengal, India 741246

CS3201 Programming and Data Structures II
March 27, 2025

Trie

The word ‘trie’ comes from the middle letters of ‘re**trie**val’.

Hence, its original pronunciation is trii as in re-trii-val.

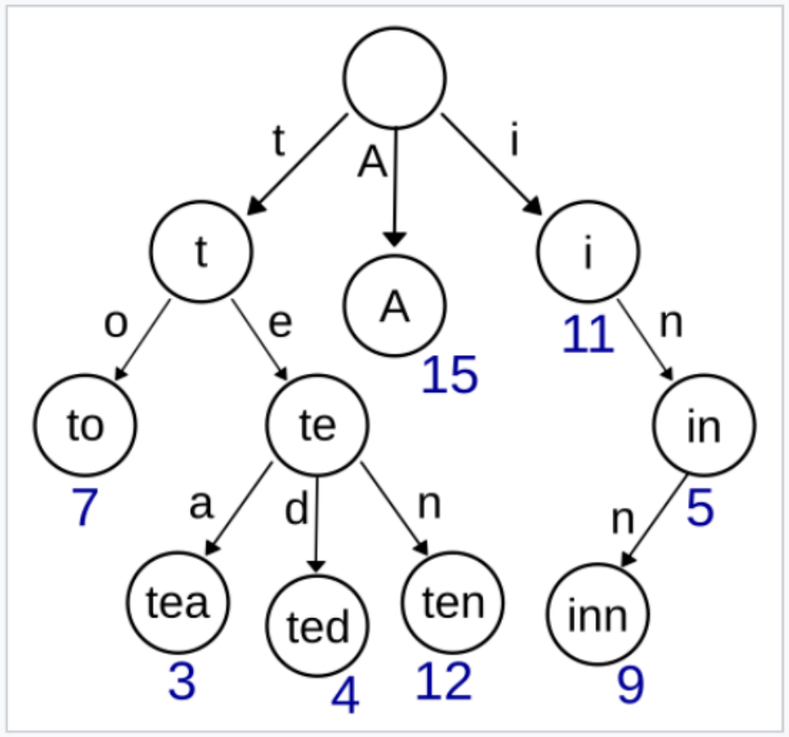
However, many people pronounce it ‘try’.

We can pronounce it either way.

Trie (contd.)

Tries are search trees specialised for ‘information retrieval’.

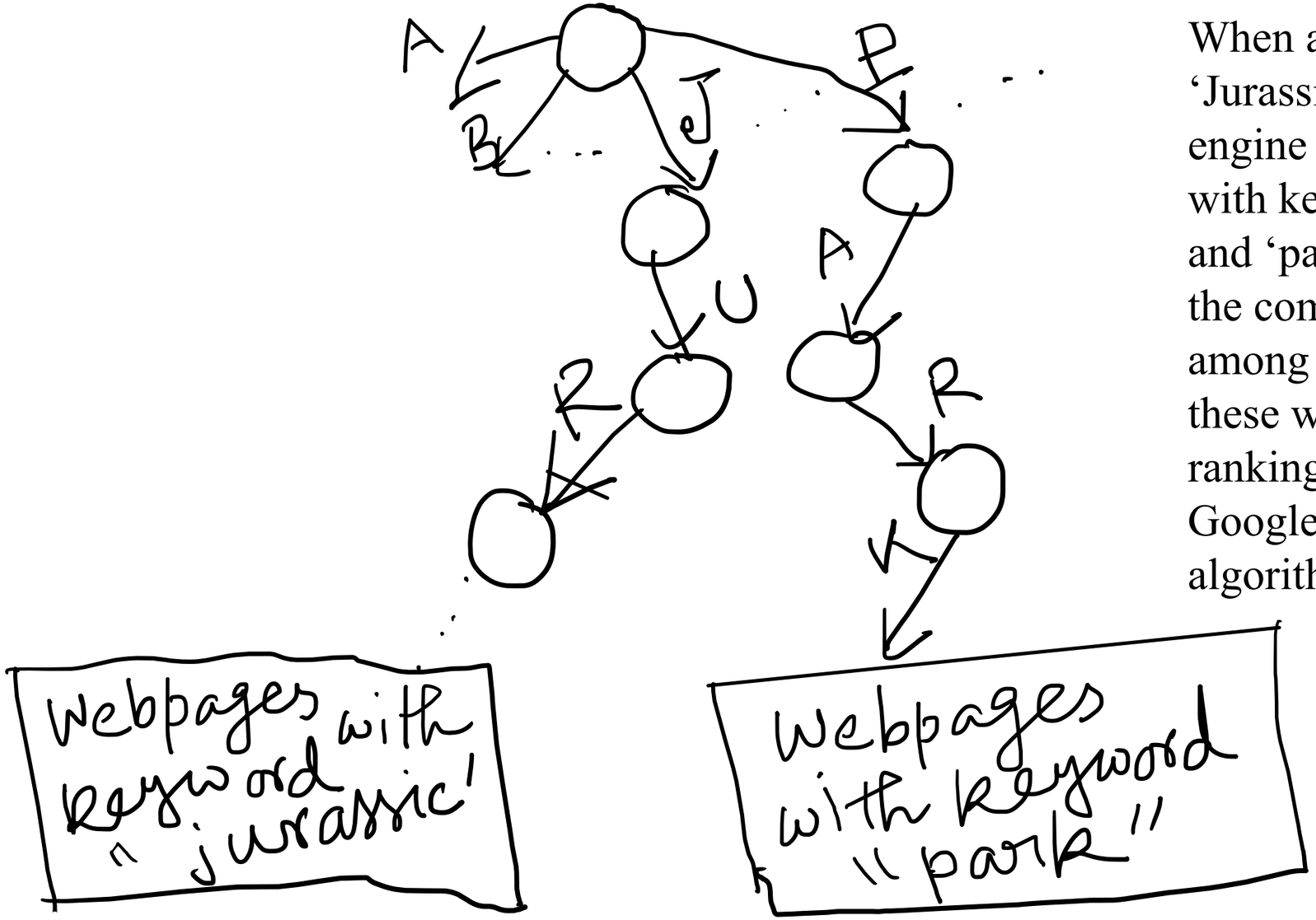
The nodes in a trie do not store a key, instead the node’s position determines the associated key.



A trie for keys "A", "to", "tea", "ted", "ten", "i", "in", and "inn". Each complete English word has an arbitrary integer value associated with it.

https://en.wikipedia.org/wiki/Trie

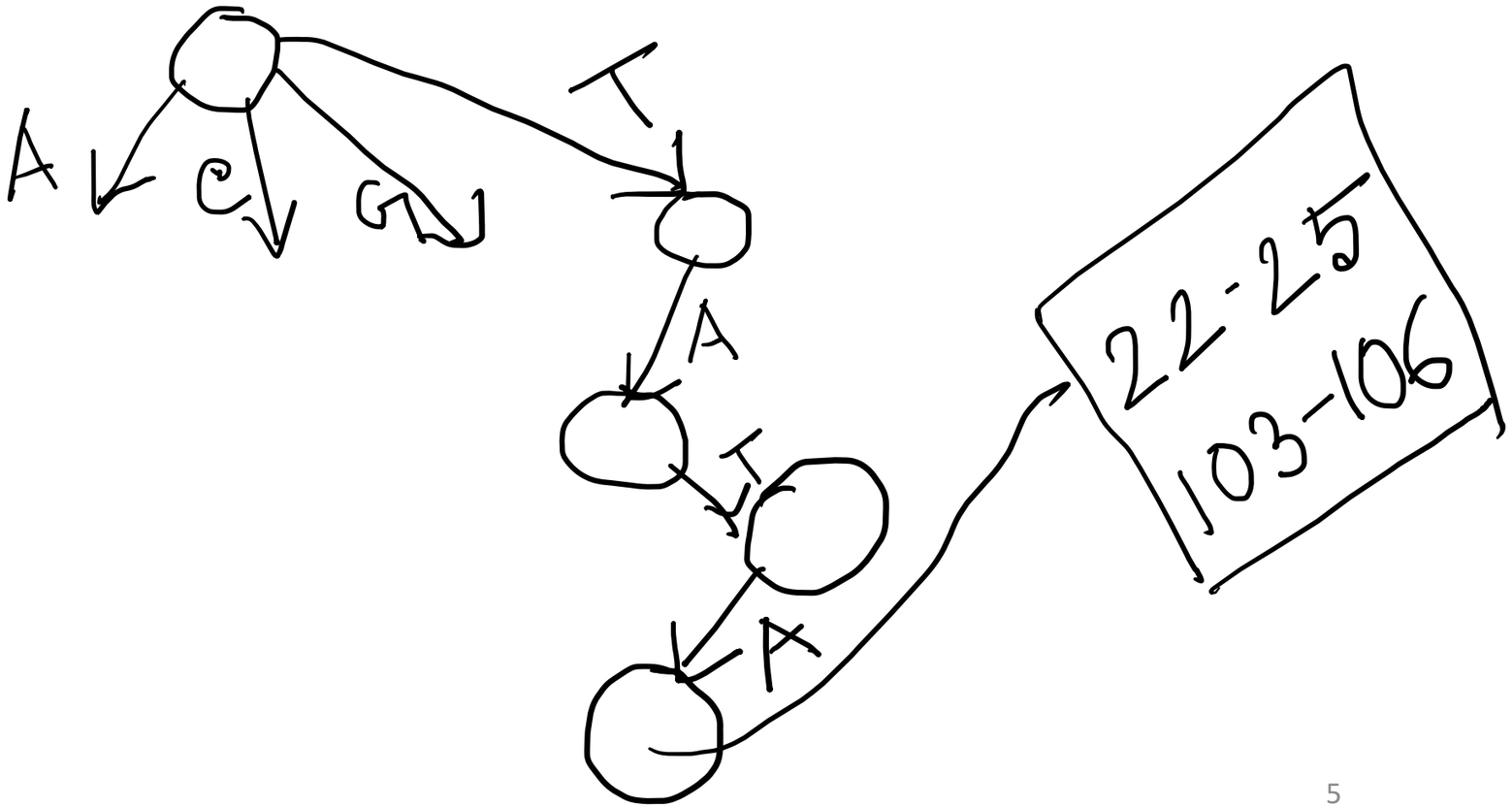
Application of Trie in Web Search Engines



When a user searches for ‘Jurassic park’, the search engine finds the webpages with keywords ‘Jurassic’ and ‘park’, then identifies the common webpages among them, and rank these webpages with a ranking algorithm, such as Google’s PageRank algorithm.

Application of Trie for DNA Sequence Alignment

The famous BLAST software uses tries to store the positions of particular subsequences.



The Minimum Spanning Tree (MST) Problem

Given a connected, weighted, undirected graph, find a subset of edges that

- connects all the nodes (a spanning tree) and
- minimises the total weight (a minimum spanning tree).

The MST Problem (contd.)

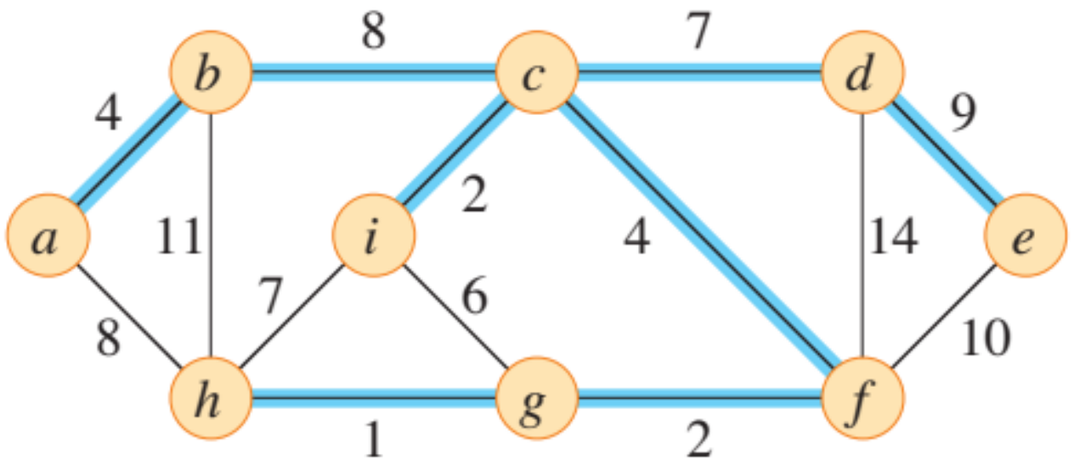


Figure 21.1 A minimum spanning tree for a connected graph. The weights on edges are shown, and the blue edges form a minimum spanning tree. The total weight of the tree shown is 37. This minimum spanning tree is not unique: removing the edge (b, c) and replacing it with the edge (a, h) yields another spanning tree with weight 37.

Algorithms for Finding MSTs

The two most widely-used algorithms for finding MSTs are:

- **Kruskal's algorithm:** Invented by Joseph Kruskal Jr. in 1956.
- **Prim's algorithm:** Invented by Prim in 1957. However, a similar algorithm was invented by Jarnik in 1930.

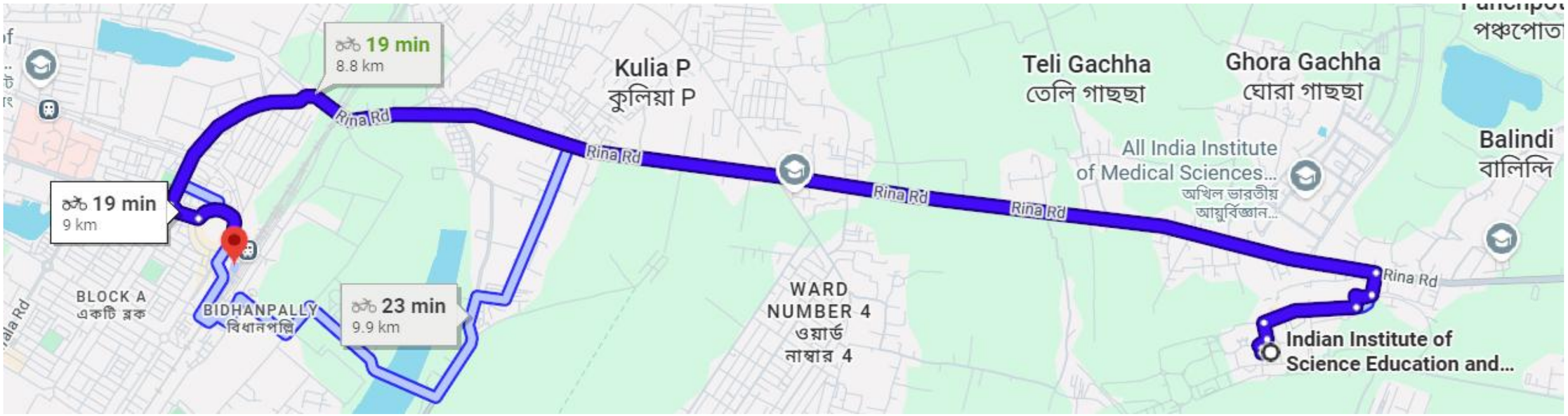
Application of MSTs

MSTs are utilised for designing:

- Taxonomy
- Biometrics
- The Internet
- Electrical grids
- Water-supply networks
- Telecom networks
- Circuit boards
- Traffic routes, etc.

https://en.wikipedia.org/wiki/Minimum_spanning_tree#Applications

Shortest Paths Problems



Destination = Kalyani Railway Station

Source = IISER Kolkata

Courtesy: Google Maps

Shortest Paths Problems (contd.)

The shortest paths problems are a collection of problems.

The Name of the Problem	**Description**
Single-source shortest paths problem	Given a source node, find the shortest paths to all other nodes
Single-pair shortest paths problem	Given a pair of nodes, find the shortest path(s) between them
Single-destination shortest paths problem	Given a destination node, find the shortest paths from other nodes to the destination node
All-pairs shortest paths problem	Find the shortest path(s) between each pair of nodes

Shortest Paths Algorithms

The Name of the Problem	**Popular Algorithms for Solving the Problem**
Single-source shortest paths problem	The Bellman-Ford algorithm Dijkstra's algorithm
All-pairs shortest paths problem	The Floyd-Warshall Algorithm Johnson's Algorithm

Shortest Paths Algorithms

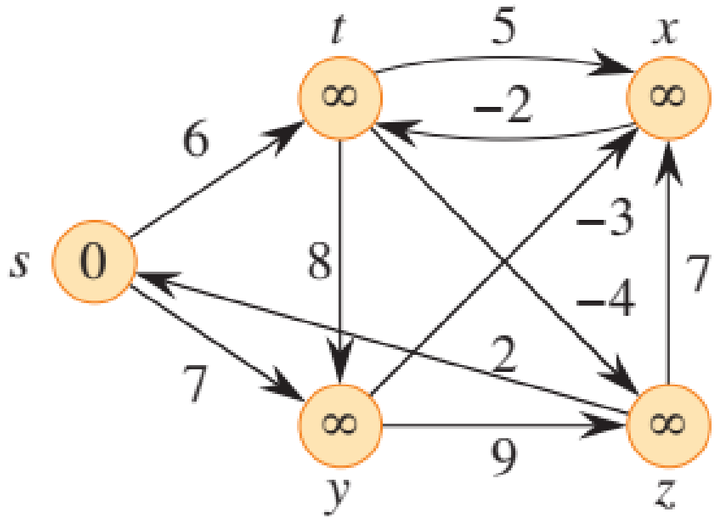
The Name of the Problem	**Popular Algorithms for Solving the Problem**
Single-source shortest paths problem	The Bellman-Ford algorithm Dijkstra's algorithm
All-pairs shortest paths problem	The Floyd-Warshall Algorithm Johnson's Algorithm

We will discuss only the Bellman-Ford algorithm.

The Bellman-Ford Algorithm

The Bellman-Ford algorithm works on any **weighted directed graphs**.

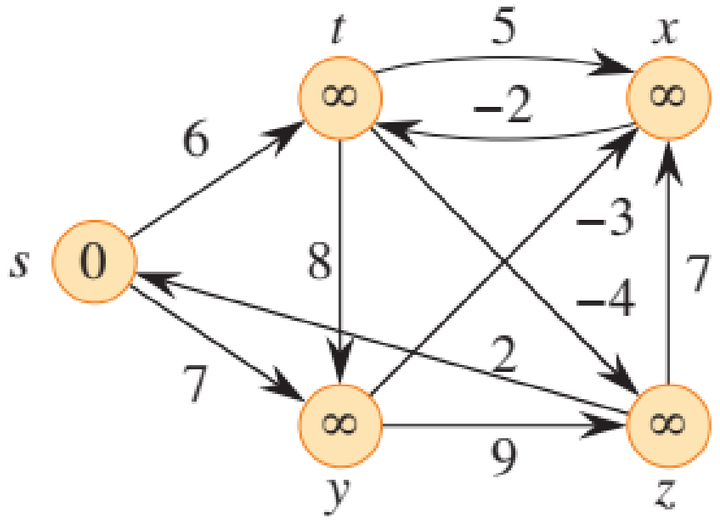
In this exemplary weighted directed graph, the content of each node is either zero (for the source node ‘s’) or infinity (for other nodes). The content represents the distance of the corresponding node from ‘s’ before the Bellman-Ford algorithm begins execution.



The Bellman-Ford Algorithm (contd.)

Even if a graph is **unweighted** directed, we can convert it to a weighted directed graph by assigning a weight of 1 to each edge.

Similarly, we can convert an **undirected** graph to a directed graph by replacing each undirected edge {u, v} with two directed edges (u, v) and (v, u).



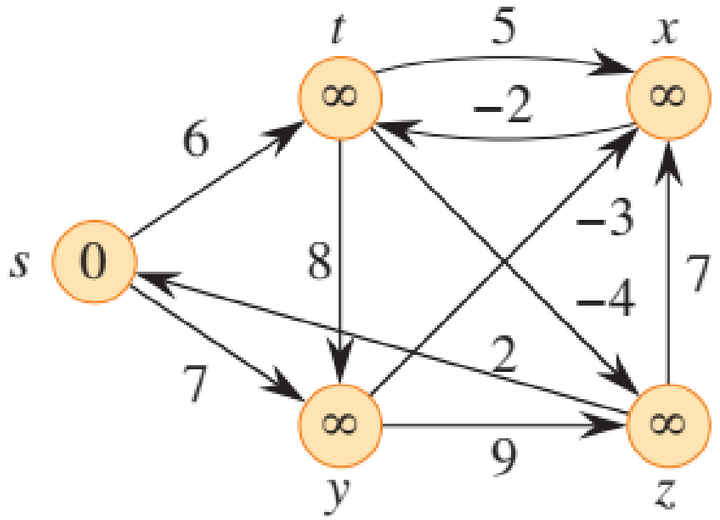
The Bellman-Ford Algorithm (contd.)

Each node should have the following structure.

Label	Distance d	Parent p

For example, node ‘s’ has the following structure.

s	d = 0	p = NULL

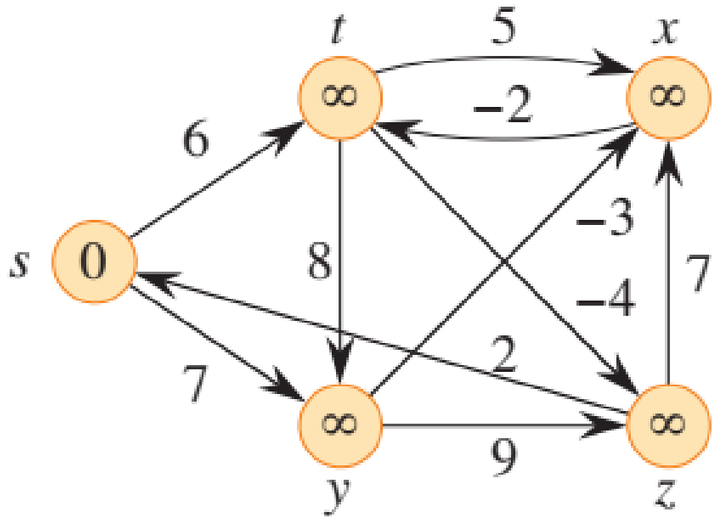


The Bellman-Ford Algorithm (contd.)

Input Parameters:

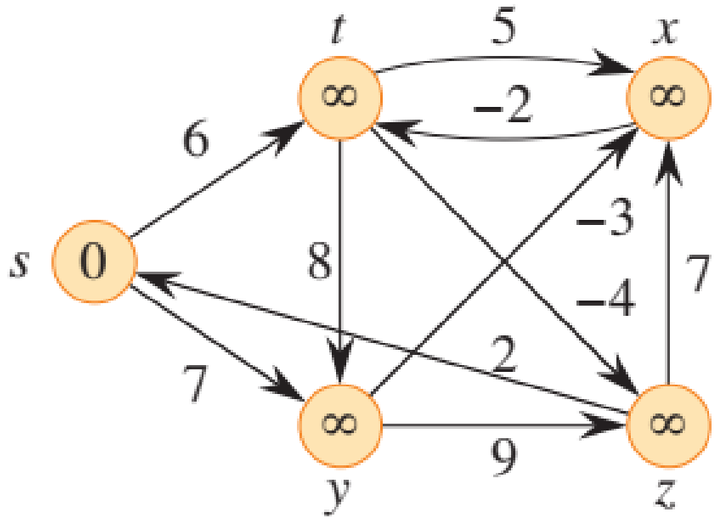
Graph G

Source node s



The Bellman-Ford Algorithm (contd.)

In addition to the node structures, graph G should also contain the **edge structures**.



The edge structures could be an array of structures where each edge is made up of the following structure.

Start node	End node	Weight w

For example, the edge (s, t) can be represented with the following structure.

start = s	end = t	w = 6

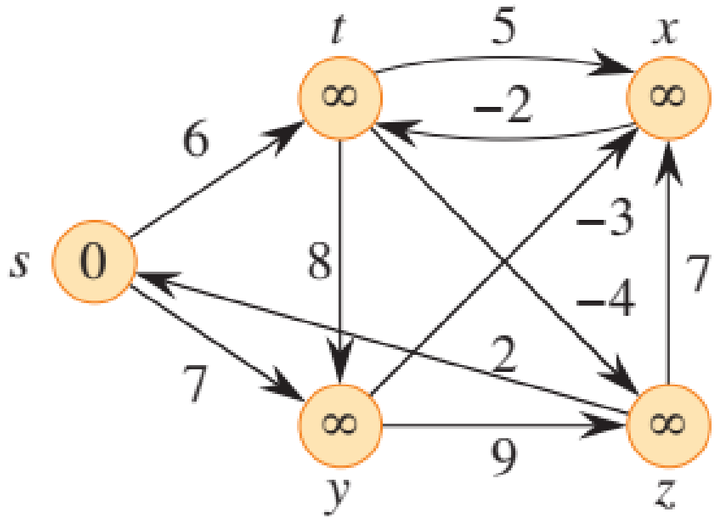
The Bellman-Ford Algorithm (contd.)

for each node v

    v.d = Infinity

    v.p = NULL

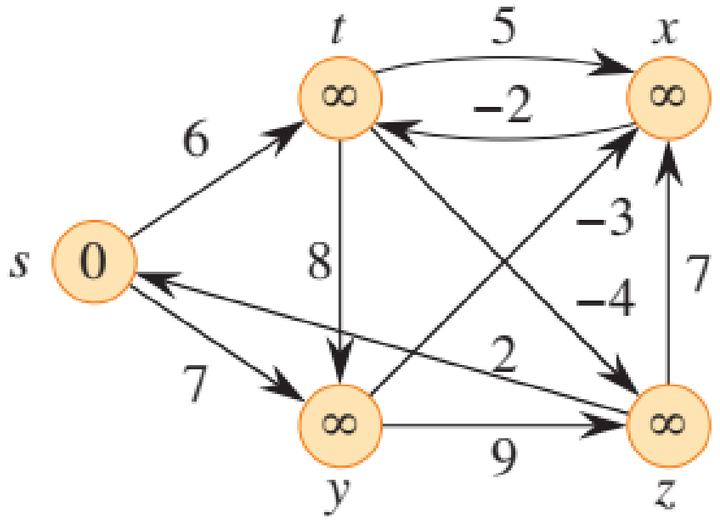
s.d = 0



As we have discussed, initialise each node with an infinite distance and a NULL parent. Then set s.d to zero since the distance of s from itself is zero.

The Bellman-Ford Algorithm (contd.)

```
for counter = 1 to (No. of nodes – 1)
  for each edge e
      u = e.start
      v = e.end
      if v.d > (u.d + e.w)
          v.d = u.d + e.w
          v.p = u
```



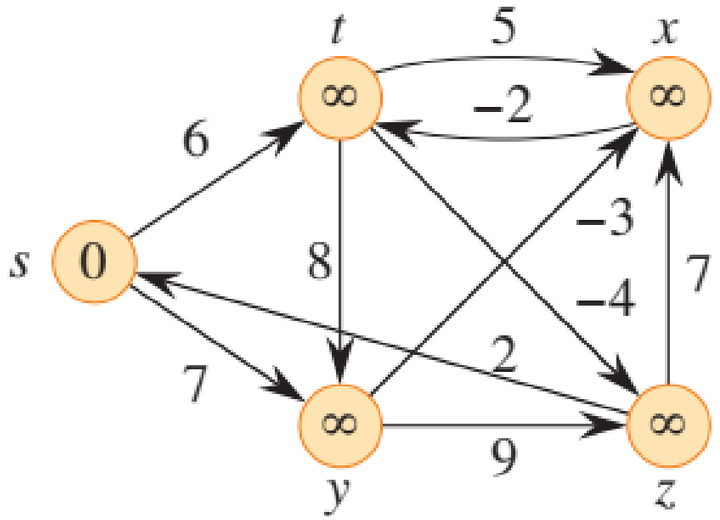
Run two nested for loops.

The outer for loop runs for (5 – 1) = 4 times.

The inner for loop runs (4 * No. of edges) times.

The Bellman-Ford Algorithm (contd.)

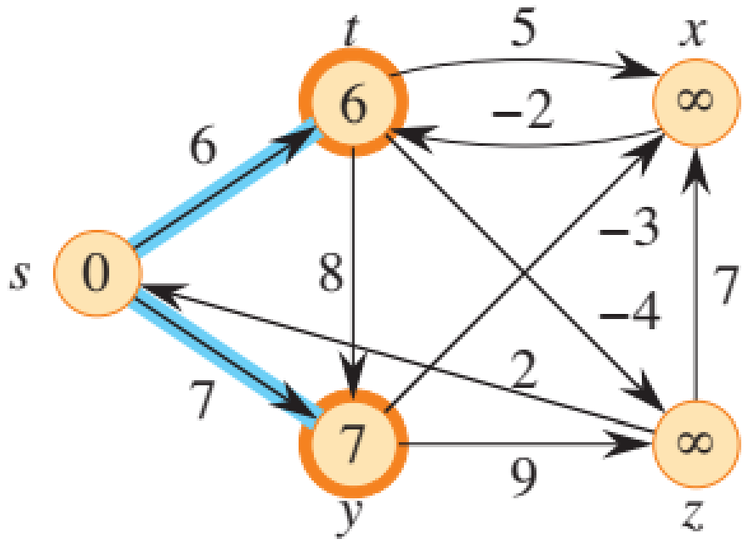
```
for counter = 1 to (No. of nodes – 1)
   for each edge e
      u = e.start
      v = e.end
      if v.d > (u.d + e.w)
         v.d = u.d + e.w
         v.p = u
```



The if block simply checks whether v's distance from s can be reduced by adding the edge (u, v) in the shortest path from s to v. If yes, the edge (u, v) is added in the shortest path from s to v. Therefore, v's distance is updated and u is assigned as the parent of v.

The Bellman-Ford Algorithm (contd.)

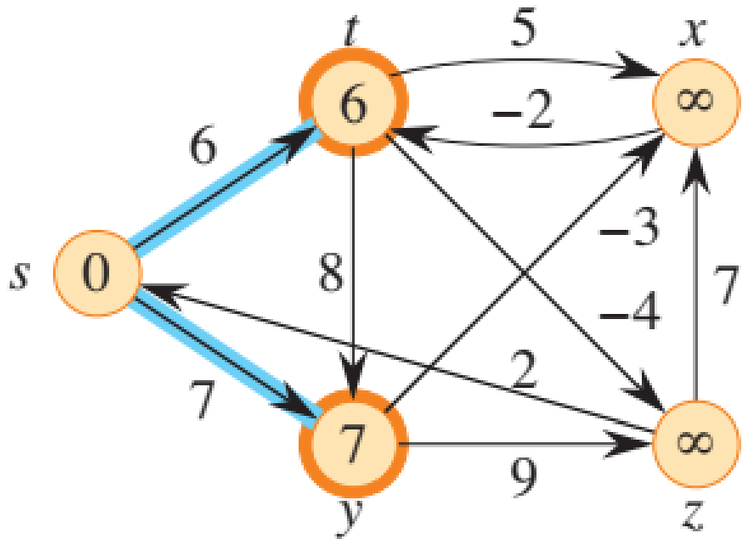
```
for counter = 1 to (No. of nodes – 1)
   for each edge e
      u = e.start
      v = e.end
      if v.d > (u.d + e.w)
         v.d = u.d + e.w
         v.p = u
```



After the **first iteration** of the outer for loop, the distances and parents of the nodes t and y are updated. The blue shades on an edge, e.g., (s, t), represent that s has been assigned as the parent of t.

The Bellman-Ford Algorithm (contd.)

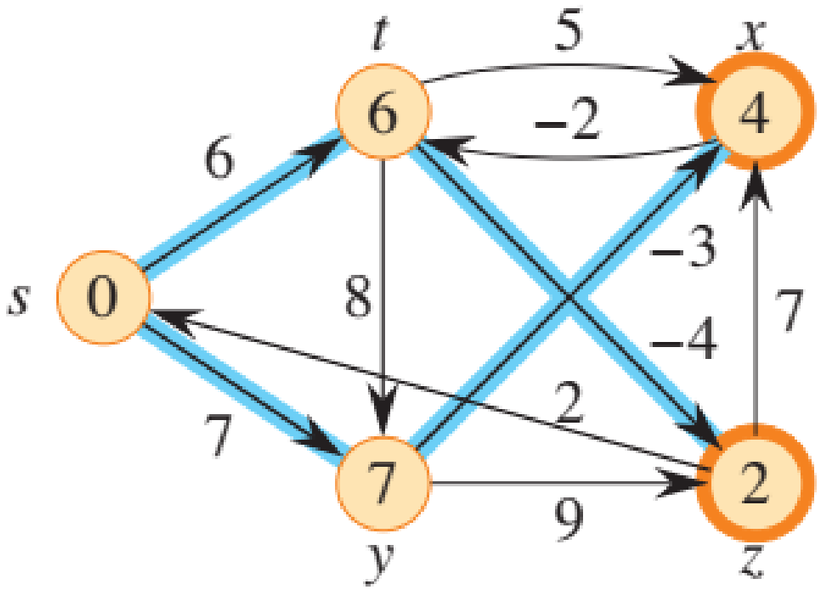
```
for counter = 1 to (No. of nodes – 1)
   for each edge e
      u = e.start
      v = e.end
      if v.d > (u.d + e.w)
         v.d = u.d + e.w
         v.p = u
```



Note: Here, we have assumed that (s, t) and (s, y) were the last two edges to be processed. If that were not the case, we could have ended up with different distances for all the nodes. However, at the end, we should obtain the correct distances for each node irrespective of the order in which we process the edges.

The Bellman-Ford Algorithm (contd.)

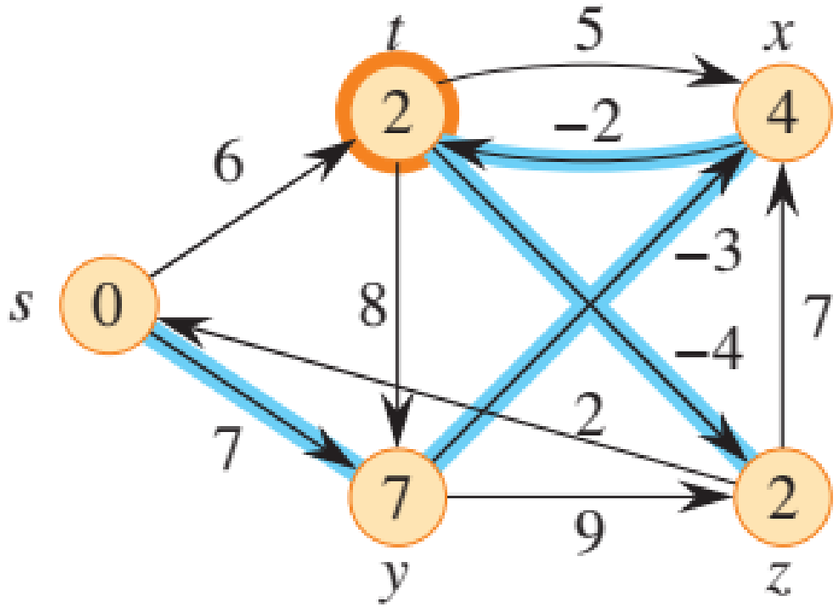
```
for counter = 1 to (No. of nodes – 1)
  for each edge e
    u = e.start
    v = e.end
    if v.d > (u.d + e.w)
      v.d = u.d + e.w
      v.p = u
```



After the **second iteration** of the outer for loop, the distances and parents of the nodes x and z are updated.

The Bellman-Ford Algorithm (contd.)

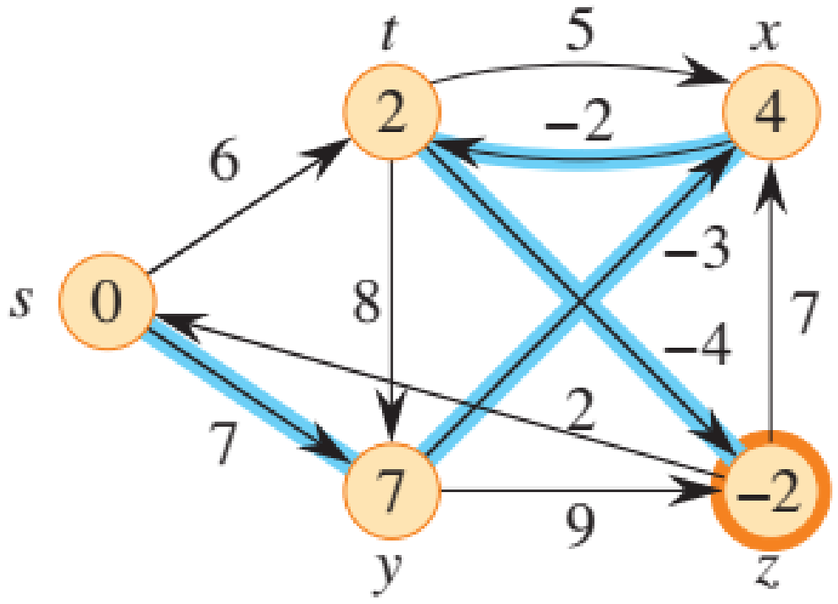
```
for counter = 1 to (No. of nodes – 1)
  for each edge e
    u = e.start
    v = e.end
    if v.d > (u.d + e.w)
      v.d = u.d + e.w
      v.p = u
```



After the **third iteration** of the outer for loop, the distance and parent of the node t are updated.

The Bellman-Ford Algorithm (contd.)

```
for counter = 1 to (No. of nodes – 1)
  for each edge e
    u = e.start
    v = e.end
    if v.d > (u.d + e.w)
      v.d = u.d + e.w
      v.p = u
```

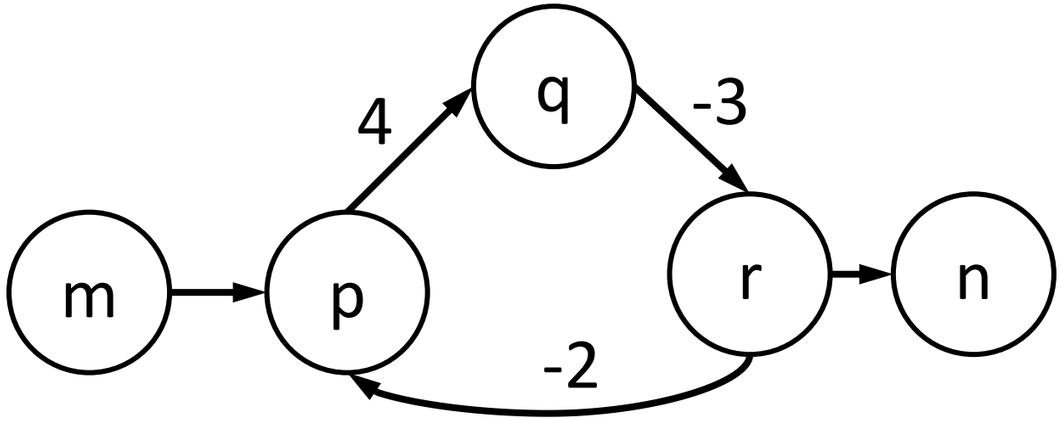


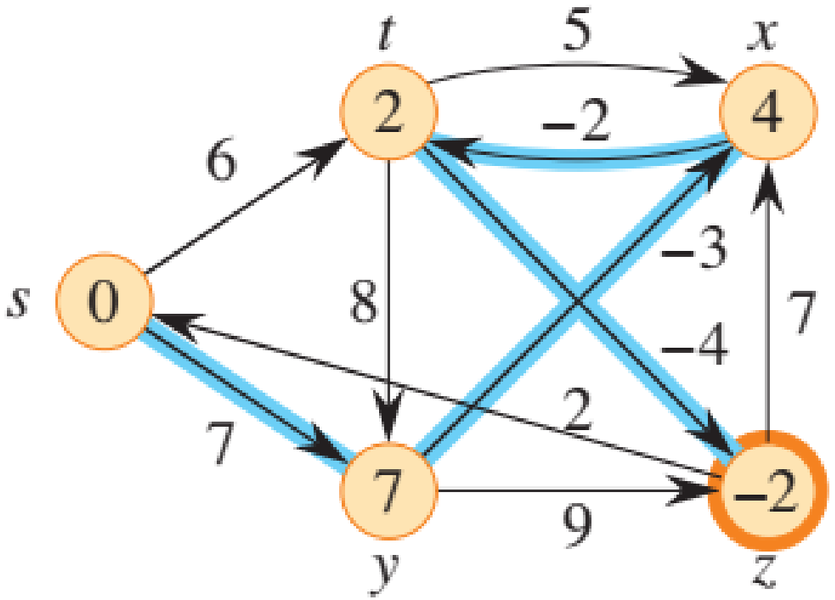
After the **fourth and final iteration** of the outer for loop, the distance and parent of the node z are updated.

The Bellman-Ford Algorithm (contd.)

The nested for loops terminate.

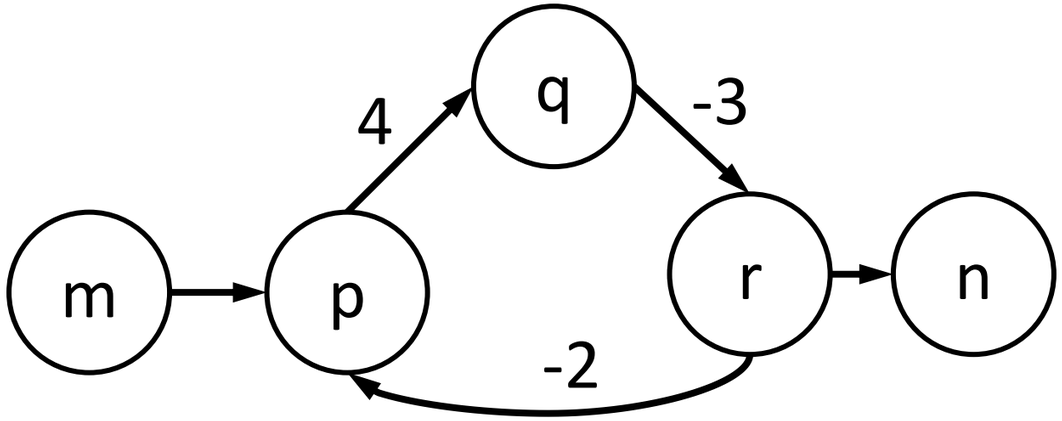
At this point, we need to check for the existence of **negative weight cycles**.

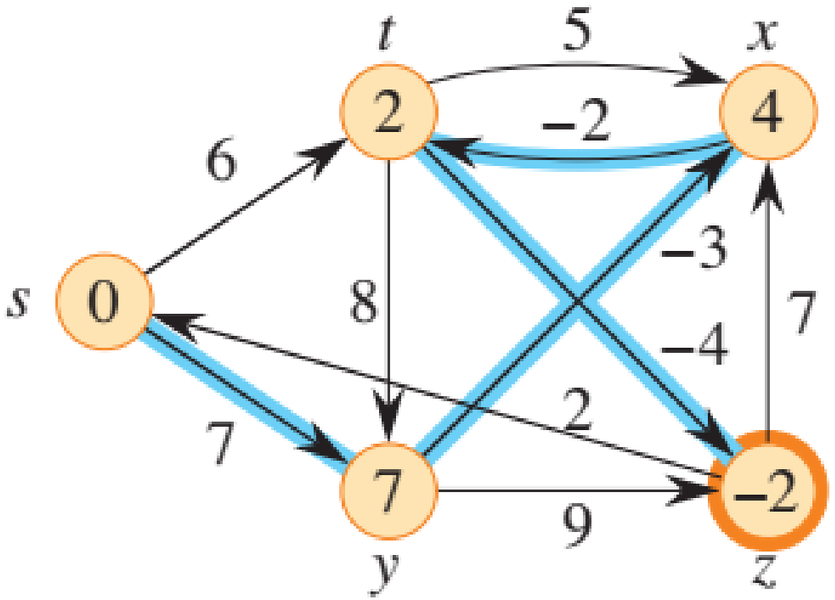




The Bellman-Ford Algorithm (contd.)

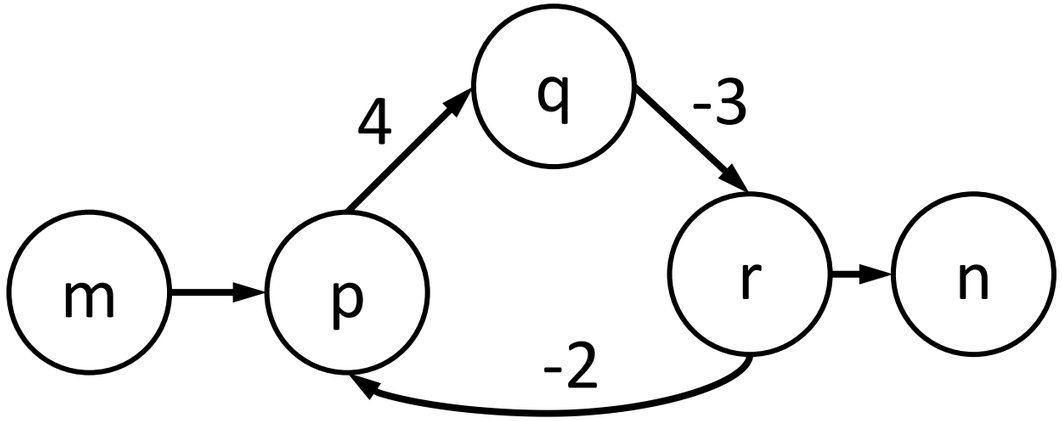
If there exists a negative-weight cycle in a path from a node m to another node n, the distance of n from m can tend to negative infinity by traversing the negative-weight cycle infinite number of times.

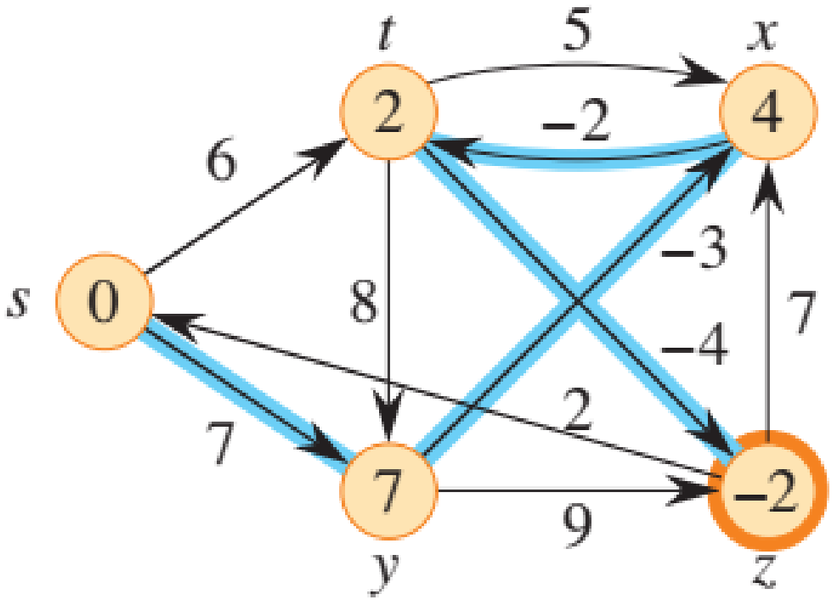




The Bellman-Ford Algorithm (contd.)

Hence, if there exists a negative-weight cycle in a graph, the notion of shortest path is **mathematically not well defined**.

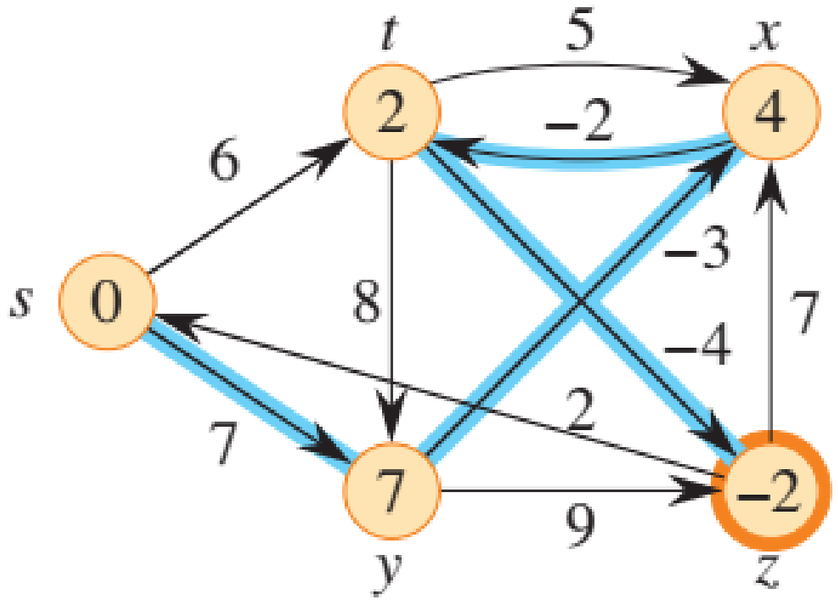




The Bellman-Ford Algorithm (contd.)

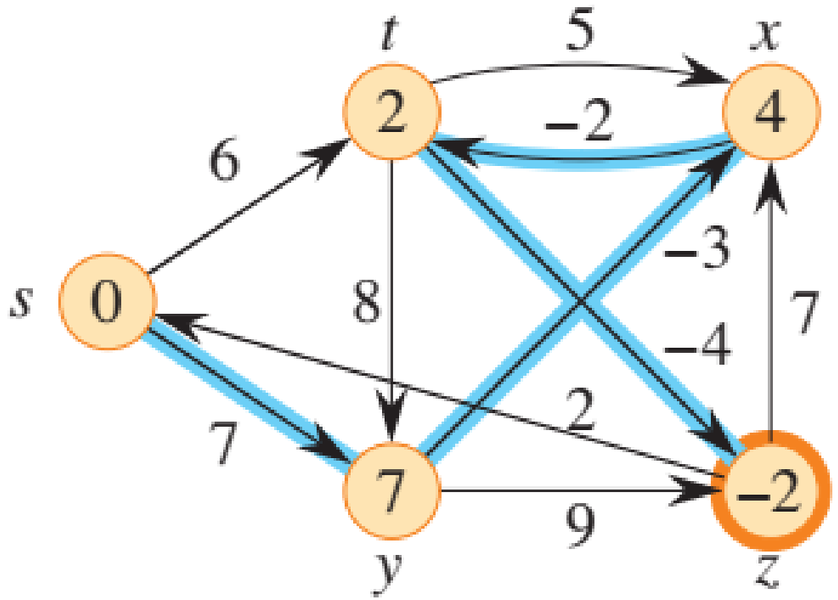
After the termination of the nested for loops, it should not be possible to reduce the distance of any node further **unless there exists any negative-weight cycles**.

Hence, we need to check whether the distances can be reduced further or not.



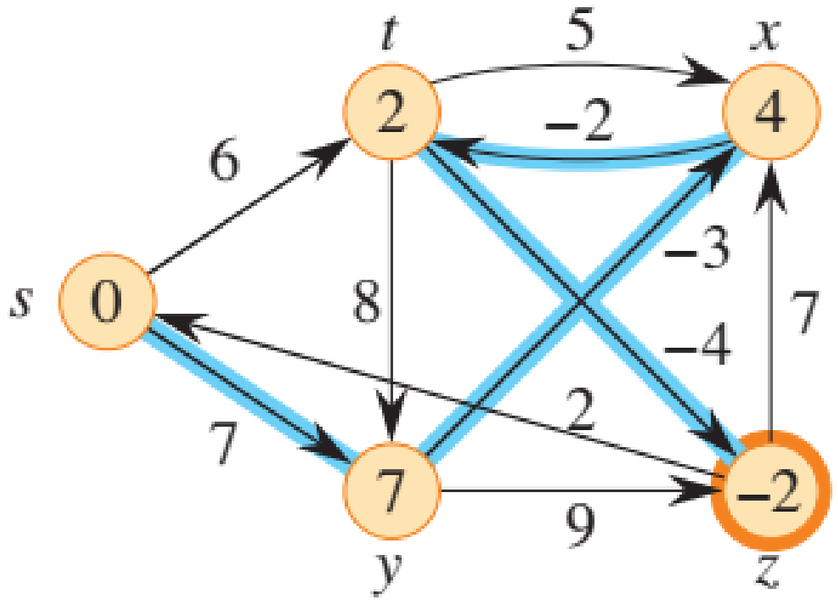
The Bellman-Ford Algorithm (contd.)

```
for each edge e
    u = e.start
    v = e.end
    if v.d > (u.d + e.w)
        print("There exists a –ve wt. cycle. Not possible
        to calculate shortest paths.")
        return FALSE
return TRUE // If this for loop is exited without entering the if block.
// We write a separate for loop to check whether any node's distance can be further reduced
or not.
```



The Bellman-Ford Algorithm (contd.)

```
for each edge e
    u = e.start
    v = e.end
    if v.d > (u.d + e.w)
        print("There exists a –ve wt. cycle. Not possible
        to calculate shortest paths.")
        return FALSE
return TRUE
```



In this example, no distances can be further reduced. It implies that there are no negative-weight cycles in the given graph. Therefore, we return TRUE. The node structures contain the correct distances and parents with which we could construct the shortest path from s to any other node.

The Bellman-Ford Algorithm: Recap

For each node, initialise the distance with infinity and parent with NULL.

Set the distance of s to zero since the distance of s from itself is zero.

Run two nested for loops to update the distances and parents of the nodes.
The outer for loop should run (No. of nodes – 1) times.
The inner for loop should run for each edge in each iteration of the outer for loop.

Check whether any distance could be further reduced. If so, there exists negative-weight cycles. Display a warning on the screen and return FALSE. Otherwise, return TRUE.

The Bellman-Ford Algorithm: Recap

For each node, initialise the distance with infinity and parent with NULL.

Set the distance of source node ‘s’ to zero since the distance of s from itself is zero.

Run two nested for loops to update the distances and parents of the nodes.
The outer for loop should run (No. of nodes – 1) times.
The inner for loop should run for each edge, in each iteration of the outer for loop.

Check whether any distance could be further reduced. If so, there exists negative-weight cycles. Display a warning on the screen and return FALSE. Otherwise, return TRUE.

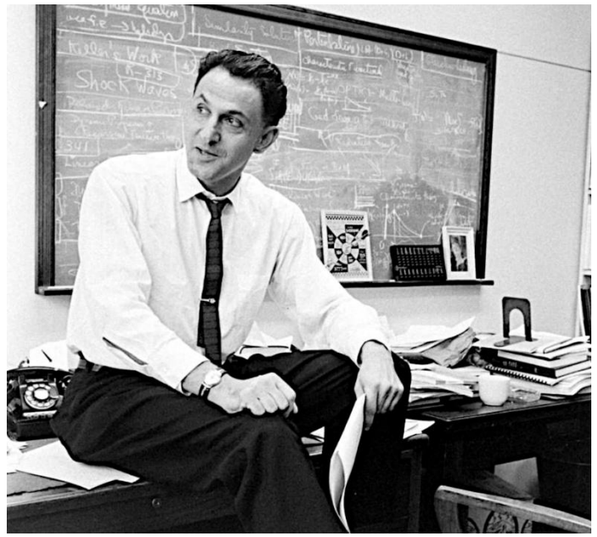
The Bellman-Ford Algorithm and Dynamic Programming

The Bellman-Ford algorithm is based on two separate algorithms designed by two American mathematicians, Richard Bellman in 1958 and Lester Ford Jr. in 1962.

The design of the Bellman-Ford algorithm follows a particular algorithmic design 'paradigm' which was invented by **Richard Bellman** in 1955 and published in 1957.

The algorithmic design paradigm is known as **dynamic programming (DP)**.

Richard Bellman was a dynamic scientist as well. You may read his autobiography 'Eye of the Hurricane'.



Chapters 14 and 22 Chapter Notes, CLRS 4th Edition
https://web.stanford.edu/class/archive/cs/cs161/cs161.1172/CS161Lecture12.pdf
Photo courtesy: https://alexxthiery.github.io/notes/HJB/HJB.html

What is an Algorithmic Design Paradigm

The word '**paradigm**' means a pattern.

An **algorithm** is a finite sequence of mathematically well-defined instructions that solves a mathematically well-defined computational problem. It was invented by the ninth-century Persian mathematician **Al-Khwarizmi** in his books on the **Indian mathematical system.**

Two or more algorithms are said to follow the same **algorithmic design paradigm** if their instruction sets have some common patterns.

https://en.wikipedia.org/wiki/Algorithm

The Dynamic Programming (DP) Paradigm

Given a computational problem, we can design a DP algorithm to solve it only if the problem has the following two properties.

Property 1 **'Optimal Substructure'**: An optimal solution to the problem can be found by

- dividing the problem into subproblems,
- finding the optimal solutions to each subproblem, and
- combining the optimal solutions of the subproblems.

Property 2 **'Overlapping Subproblems'**: If the subproblems share some common subsubproblems.

The Dynamic Programming (DP) Paradigm (contd.)

If the problem has these properties, when designing an algorithm, we must ensure that each common subsubproblem is solved **exactly once** and its solution is remembered or '**memo**ised'.

A Case Study

There are 3 class tests on the same day. You need to prepare for all of them. It's a **problem**.

A Case Study (contd.)

There are 3 class tests on the same day. You need to prepare for all of them. It's a **problem**.

Solution. Design an algorithm that will maximise your preparation for all three class tests.

A Case Study (contd.)

There are 3 class tests on the same day. You need to prepare for all of them. It's a **problem**.

Step 1 for designing an algorithm: Analyse the problem to understand its underlying properties.

A Case Study (contd.)

Optimal Substructure: If you maximise your preparation for each class test, then your preparation is also maximised for all class tests.

Overlapping Subproblems: Are there any common topics among the three class tests?

- If not, then we should design a **divide-and-conquer** algorithm, another algorithmic design paradigm [1].
- If yes, then we can design a DP algorithm ensuring that the common topics are prepared exactly once.

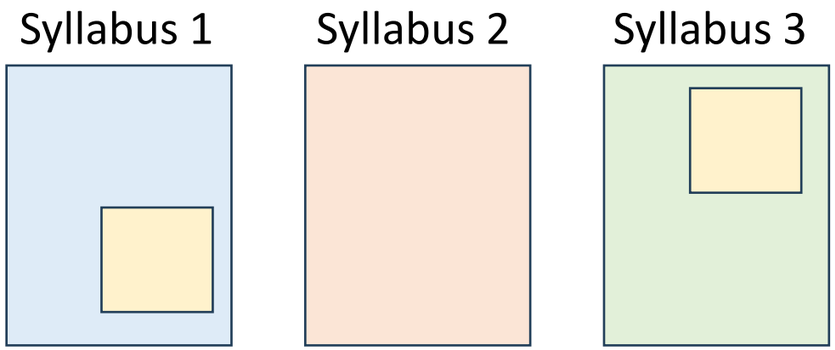
Suppose there are some common topics. Let us design a DP algorithm.

[1] Section 2.3.1 'The divide-and-conquer method', CLRS 4th Edition

A Case Study (contd.)

Algorithm

Identify the common topics (the orange boxes inside).

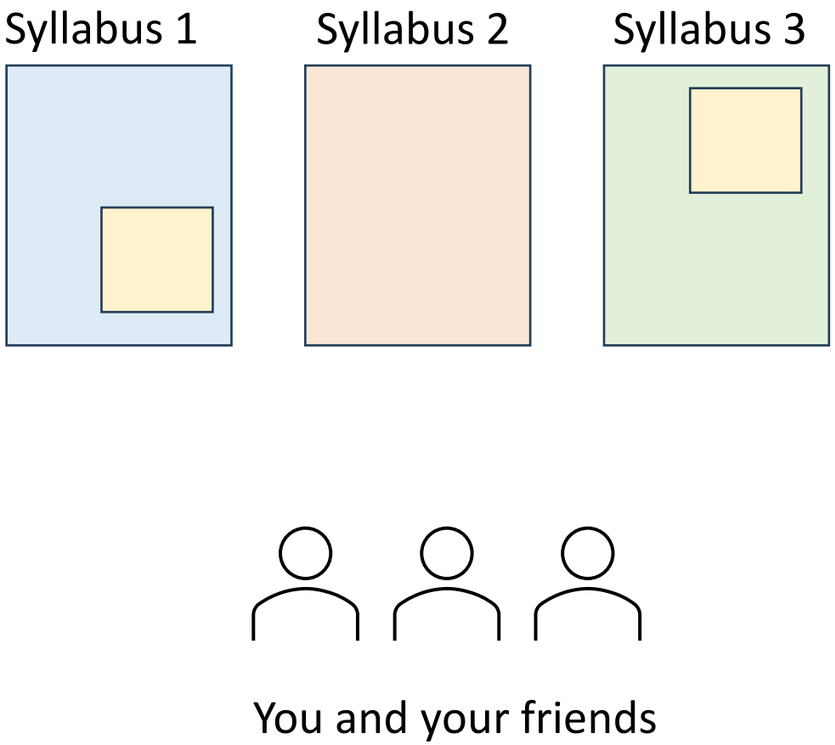


A Case Study (contd.)

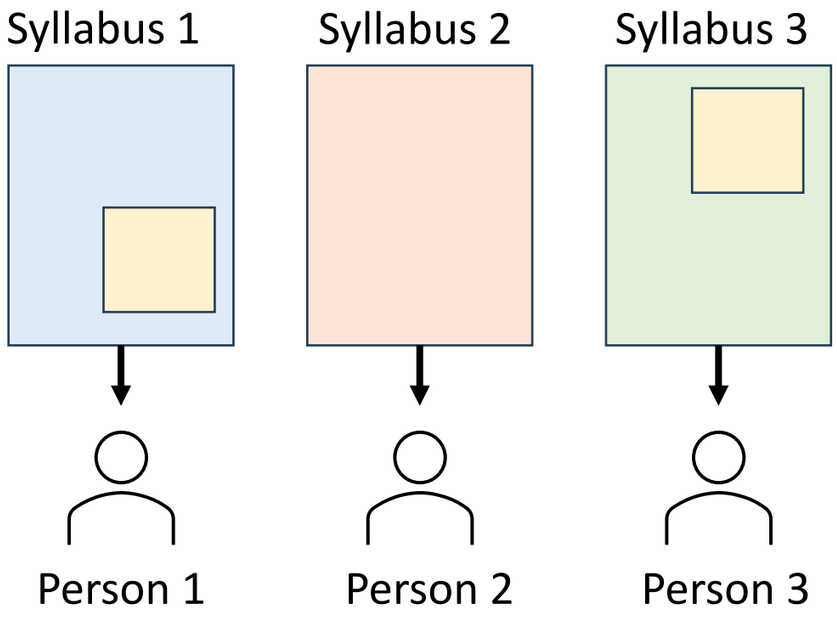
Algorithm

Identify the common topics (the orange boxes inside).

Bring in (3 – 1) = 2 friends. // 1 deducted for yourself



A Case Study (contd.)



Algorithm

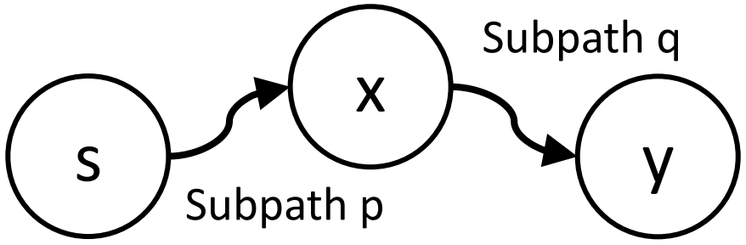
Identify the common topics (the orange boxes inside).

Bring in (3 – 1) = 2 friends.

```
for syllabus i = 1 to 3
    for each topic t
      if t has already been covered by someone other than person i
         Person i gets the lesson from that person.
     else
         Person i prepares for t and teaches it to others.
```

How Is Bellman-Ford a DP Algorithm?

The Bellman-Ford algorithm exploits the optimal substructure and overlapping subproblems property of the single-source shortest paths problem.



Optimal Substructure

If a shortest path from node s to node y consists of the subpath p from s to x and the subpath q from x to y, then

- subpath p is a shortest path from s to x, and
- subpath q is a shortest path from x to y. (Prove by contradiction)

Therefore, the problem of finding a shortest path from s to y via x can be divided into:

- finding a shortest path from s to x, and
- finding a shortest path from x to y.

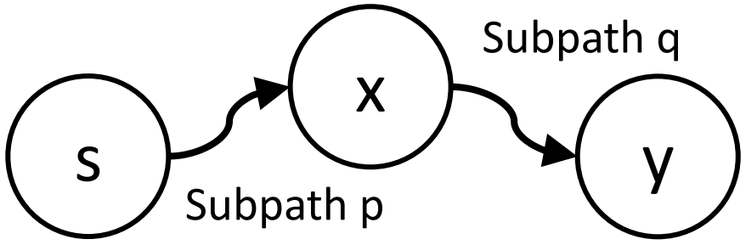
https://web.stanford.edu/class/archive/cs/cs161/cs161.1172/CS161Lecture12.pdf

How Is Bellman-Ford a DP Algorithm? (contd.)

The Bellman-Ford algorithm exploits the optimal substructure and overlapping subproblems property of the single-source shortest paths problem.

Overlapping Subproblems

Finding a shortest path from s to any node v via x has a common subproblem, which is to find a shortest path from s to x.



In the Bellman-Ford algorithm, a shortest path from s to x is **memoised** using the 'p' attribute of the nodes that lie on the path. The distances of these nodes from s are also memoised using the 'd' attribute of the nodes.

https://web.stanford.edu/class/archive/cs/cs161/cs161.1172/CS161Lecture12.pdf

Why is Dynamic Programming called ‘Dynamic Programming’?

The DP algorithms are mainly suitable for solving **optimisation problems**, e.g., the shortest paths problems where we need to find a path that minimises the distance between two nodes.

The **linear programming** methods, which are older than DP algorithms, have also been suitable for solving optimisation problems. Hence, Bellman used the term ‘programming’ for his algorithmic design paradigm.

Further, Bellman added the adjective **‘dynamic’** to represent the **temporal dependencies** between different elements of the DP paradigm, e.g.,
if a common subsubproblem has already been solved while solving a subproblem, we do not solve the subsubproblem again when solving other subproblems.

Practice Problem

Given a directed **unweighted** graph, write a C program that uses dynamic programming to print a shortest path from node 0 to node 3, if any.

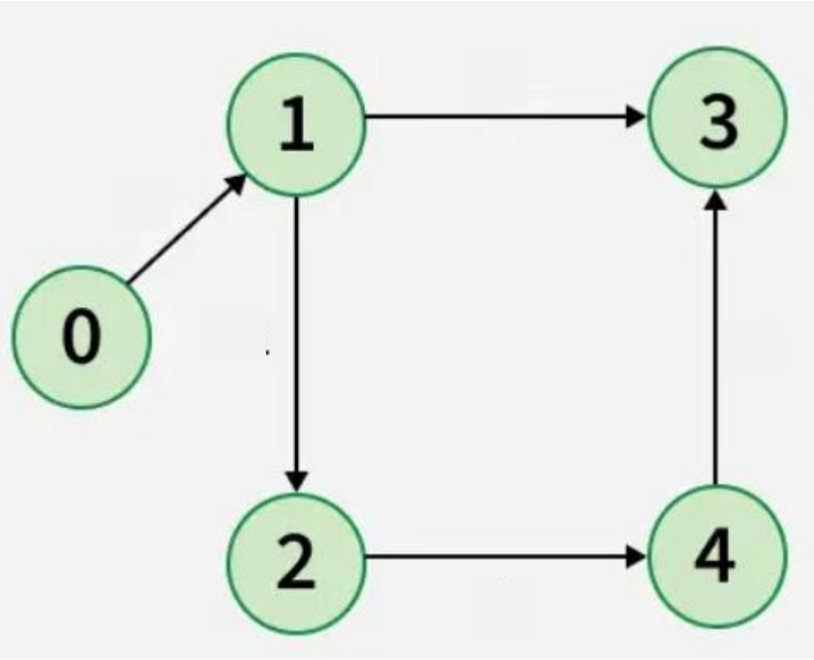


Figure adapted from https://www.geeksforgeeks.org/dsa/bellman-ford-algorithm-dp-23/

That concludes SP's Part of the CS3201 Syllabus

Linked lists: Singly and doubly linked list (non-circular and circular). Stack and queue using linked list. [10 hours]

Trees: Binary tree - traversal (inorder, preorder, postorder), searching (DFS, BFS), Search tree: Binary Search Tree - insertion, deletion, searching. [8 hours]

Applications of tree: Heap - insertion, deletion and sorting. Priority queue revisited. Huffman encoding. [8 hours]

Advanced trees (brief introduction only, no details): Height balanced trees (AVL, Red-black), B-tree, B+ tree, TRIE, minimum spanning trees, shortest paths. [6 hours]

Hashing: Hash functions: properties and standards hash functions; Collision and resolution techniques - Probing (linear and quadratic) and Chaining (linear and coalesced). [3 hours]

Advanced topic: Dynamic programming. [1 hours]

What’s Next

Wednesday, April 1, 2026: **Class Test 2**

Friday, April 3, 2026: Good Friday. No classes.

Wednesday-Friday, April 8-10, 2026: Kripa Sir will start discussing ‘heaps’. **Heaps** are used by all modern operating systems, such as Linux and Android, for scheduling processes.

Wednesday, April 15, 2026: **Class Test 3**

Friday, April 17, 2026: Buffer. Last day of classes.

Wednesday, April 22, 2026: CS3201 End-Sem Exam **(Tentative)**.

Syllabus Load Balancing for Exams

Class Test 2	(Tree traversals + AVL trees) = 25 marks
Class Test 3	(Tree traversals + red-black trees) 12.5 marks + Kripa sir's 12.5 marks = 25 marks
End-Sem Exam	(Tree traversals + BFS,DFS + ordinary BSTs + B,B^+-trees' node structures + Shortest Paths + DP) 25 marks + Kripa sir's 25 marks = 50 marks

Note: Your best 2 of 3 class tests will be considered.

Thank You